

BUỔI 06

Mở lớp học

CourseClass • @ManyToOne • Dữ liệu vận hành

Course

Room

ScheduleSlot

Teacher × 2





1

Thiết kế lớp học thực tế

Hiểu CourseClass là dữ liệu **VẬN HÀNH** —
kết nối Course, Room, ScheduleSlot và
Teacher thành một lớp chạy thật



2

Map quan hệ @ManyToOne

Một entity có nhiều khóa ngoại cùng lúc.
CourseClass có 5 @ManyToOne: course,
room, slot, mainTeacher, assistantTeacher



3

Phân biệt Course vs CourseClass

Course = danh mục khóa học (ít thay đổi).
CourseClass = lớp đang vận hành thực tế
(có lịch, phòng, sĩ số, trạng thái)



Course vs CourseClass — Sự khác biệt then chốt

Buổi 06 • Mở lớp học



Course

Danh mục khóa học



Danh mục cố định, ít thay đổi



Tên khóa, mã khóa, cấp độ



Được tái sử dụng nhiều lần



Không có lịch học, phòng, giáo viên



Ví dụ: Toán lớp 10, Anh văn cơ bản

tạo ra



CourseClass

Lớp học vận hành thực tế



Gắn với phòng học cụ thể (Room)



Có lịch học rõ ràng (ScheduleSlot)



Có giáo viên + trợ giảng (Teacher)



Có trạng thái: OPEN / ONGOING / CLOSED

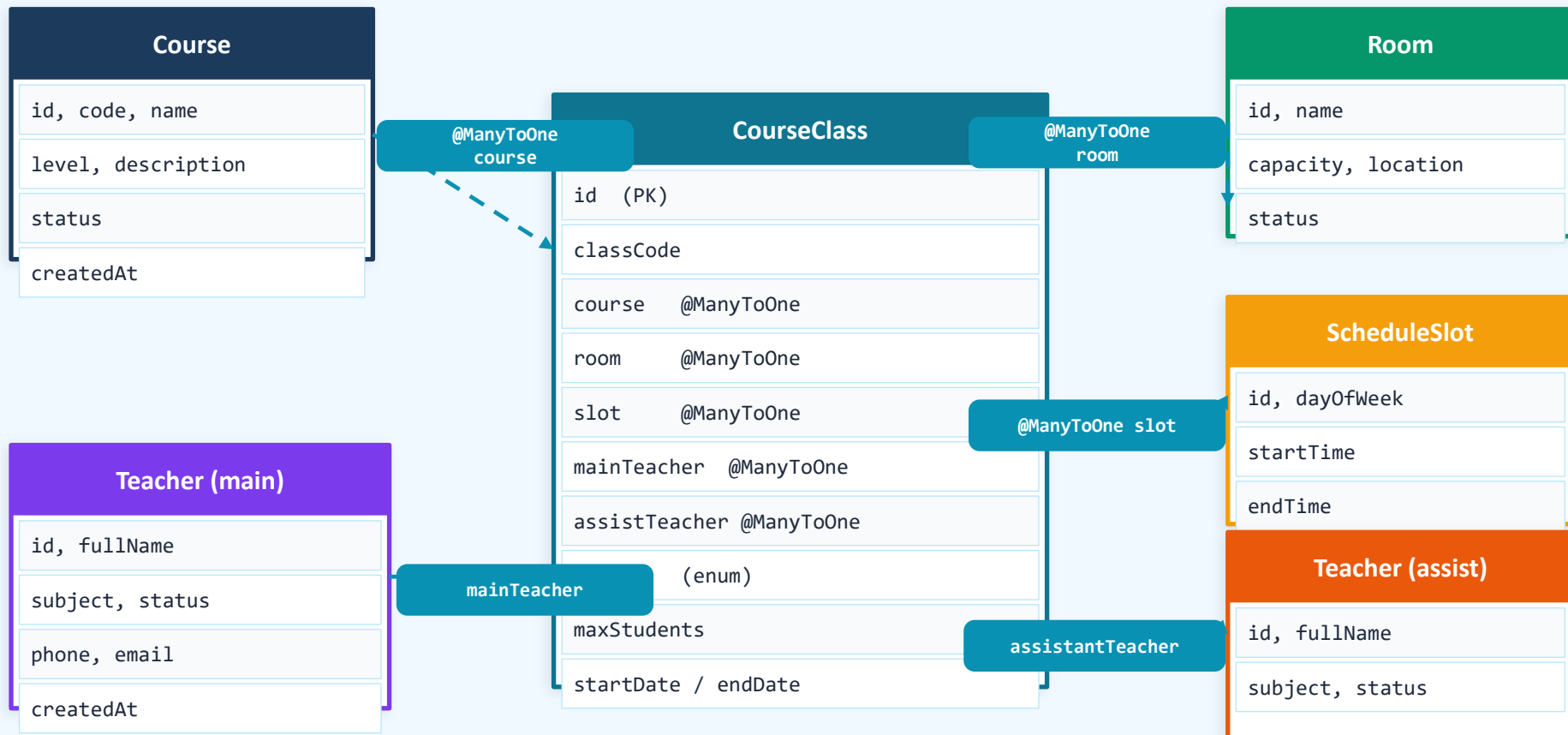


Ví dụ: Lớp Toán10-A, Phòng 201, T2-T4



Sơ đồ quan hệ — CourseClass & @ManyToOne

Buổi 06 • Mở lớp học

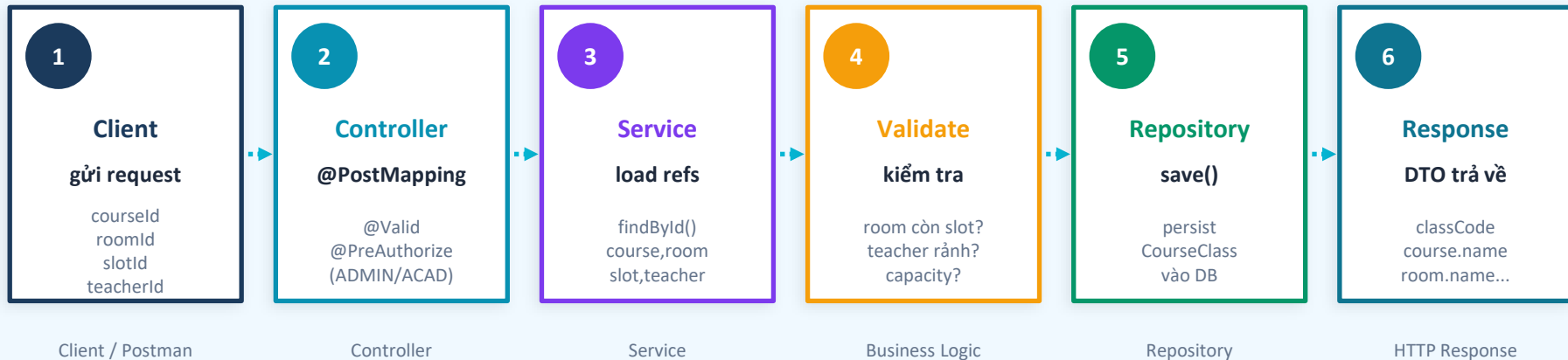


2 khóa ngoại đều trỏ về bảng Teacher — dùng `@JoinColumn(name="main_teacher_id")` và `@JoinColumn(name="assistant_teacher_id")` để phân biệt

+ Workflow A — Mở lớp học (POST /api/course-classes)

Buổi 06 • Mở lớp học

Admin hoặc Academic Staff điền form → API tạo CourseClass với đầy đủ dữ liệu tham chiếu:



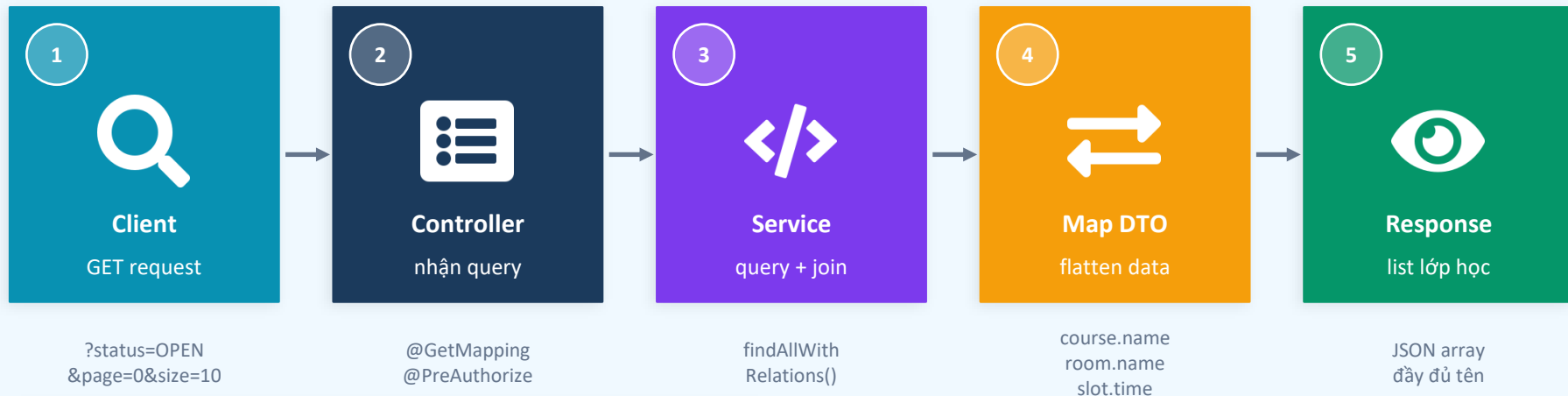
Điểm cần nhớ:

- Service phải load đủ tất cả tham chiếu (Course, Room, Slot, Teacher) trước khi gọi save() — không dùng ID raw.
- Phải validate: phòng có còn slot không, giáo viên có bị trùng lịch không, sĩ số có vượt capacity không.

Workflow B — Hiển thị lớp học (GET /api/course-classes)

Buổi 06 • Mở lớp học

Frontend cần hiển thị đầy đủ thông tin lớp: tên khóa, phòng, lịch, giáo viên — không chỉ ID:



Response DTO mẫu

```
{ "classCode": "TOAN10-A",  
  "course": { "name": "Toán lớp 10" },  
  "room": { "name": "Phòng 201" },  
  "slot": { "day": "T2-T4", "time": "18:00" },  
  "mainTeacher": { "name": "Nguyễn Văn A" } }
```

⚠ Tại sao cần map DTO?

- Không trả thẳng entity — lộ DB schema
- Lazy-load relation gây N+1 query nếu không cẩn thận
- Frontend cần tên, không cần ID của Room, Slot
- Dùng @EntityGraph hoặc JOIN FETCH để tải đủ



Demo API — Chuỗi endpoint trên lớp

Buổi 06 • Mở lớp học

#	Method	Endpoint	Quyền	Mục đích demo
1	GET	/api/reference/rooms	ANY	Tải dropdown Room cho form
2	GET	/api/reference/schedule-slots	ANY	Tải dropdown ScheduleSlot
3	GET	/api/reference/teachers	ANY	Tải dropdown Teacher
4	POST	/api/course-classes	ADMIN / ACAD	Tạo lớp học — demo validate & save
5	GET	/api/course-classes	ANY	Xem danh sách lớp + full relation
6	GET	/api/course-classes/{id}	ANY	Xem chi tiết 1 lớp
7	POST	/api/course-classes (thiếu ref)	—	Demo lỗi 400 khi courseId không tồn tại



Thứ tự demo: tải ref data → tạo lớp thành công → xem list → xem chi tiết → demo lỗi thiếu dữ liệu tham chiếu



Bài tập cuối buổi 6

01

Filter lớp theo status

Thêm query param
`?status=OPEN|ONGOING|CLOSED` vào
`GET /api/course-classes`. Dùng
`@RequestParam(required=false)`.

02

Kiểm tra sĩ số tối đa

Khi tạo lớp, validate $\text{maxStudents} \leq \text{room.capacity}$. Nếu vượt $\rightarrow$ trả về 400
với message rõ ràng.

03

Class code tự động

Service sinh classCode theo
convention: `{courseCode}-{roomName}-{year}`. Ví dụ: TOAN10-
P201-2025.

"CourseClass là nơi danh mục trở thành thực tế — thiếu một relation là lớp mất ý nghĩa vận hành."